Feedback Control Systems

EE-3004

Complex Engineering Problem (CEP)

Spring 2026

Design a Compensator for the control of a Ball and Balancer System

(Phase 2)

Submitted by:

Muhammad Ibrahim (23i – 6090)

Mohammad Ahtesam (23i – 6060)

Course Instructor:

Engr. Maria Nasir

Lab Instructor:

Engr. Abu Baker

National University of Computer and Emerging Sciences, Islamabad

CEP Statement:

Design a Compensator for the control of Ball and Balancer System:

1. **Analyze a given system for stability, performance, and robustness using s-domain techniques (Routh-array and Root-locus) and frequency-domain techniques (Bode- and Nyquist- plots).**
2. **Recommend a controller including a compensator for an open-loop system to meet closed-loop requirements of stability, performance, and robustness.**

For the Assessment of CLOs:

CLO-2: Analyze a given system for stability, performance, and robustness using s-domain techniques (Routh-array and Root-locus) and frequency-domain techniques (Bode- and Nyquist- plots).

CLO-3: Recommend a controller including a compensator for an open-loop system to meet closed-loop requirements of stability, performance, and robustness.

Phase 1:

For the Assessment of CLO-2: Analyze a given system for stability, performance, and robustness using s-domain techniques and frequency-domain techniques.

The following simplified dynamic equation has been linearized about $\alpha = 0$ for a ball and balancer control system:

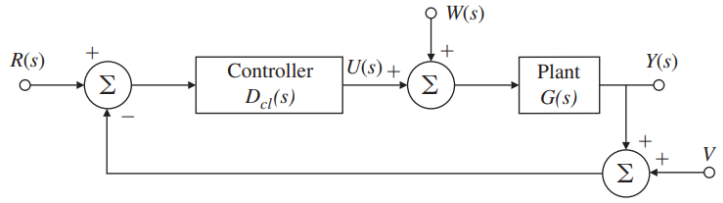
$$\left(\frac{\boldsymbol{I}}{\boldsymbol{R^2}} + \boldsymbol{m}\right)\ddot{\boldsymbol{y}} = -\boldsymbol{mg}\frac{\boldsymbol{d}}{\boldsymbol{L}}\boldsymbol{\theta}$$

In this system $y(t)$ is the output of the open loop system and $\theta(t) = u(t)$ is the control input to the system. i.e. $G(s) = \frac{Y(s)}{\theta(s)}$

For this problem, we will assume that the ball rolls without slipping and friction between the surface and ball is negligible. The constants and variables for this system are defined as follows:

$$(m) \quad mass\ of\ the\ ball \quad = \ 0.11\ kg$$
$$(R) \quad radius\ of\ the\ ball \quad = \ 0.015\ m$$
$$(d) \quad lever\ arm\ offset \quad = \ 0.03\ m$$
$$(g) \quad gravitational\ acceleration \quad = \ -9.8\ m/s^2$$
$$(L) \quad length\ of\ the\ balancer \quad = \ 1.0\ m$$
$$(I) \quad ball's\ moment\ of\ inertia \quad = \ 9.99 \times 10^{-6}\ kg.m^2$$
$$(y) \quad ball\ position\ coordinate$$
$$(alpha) \quad balancer\ angle\ coordinate$$
$$(theta) \quad servo\ gear\ angle$$

Consider the following unity feedback architecture for the closed loop system:



A reference input $R(s) = Y_{des}(s) =$ **step of <u>0.25 units</u>** is provided to the closed loop system, and the **design specifications** for the overall closed loop system are as follows:

- The overshoot should be less than 5%
- Settling time should be less than 3 seconds for a 2% settling time
- Steady-state error for reference tracking should be less than 2%

Using the differential equations given: **[9*4 = 36 Marks]**

a) Find the open loop transfer function $G(s) = \frac{Y(s)}{\theta(s)}$ of the plant.

Calculations:

Given $\left(\frac{I}{R^2} + m\right)\ddot{y} = -mg\frac{d}{L}\theta$

Taking Laplace transform on both sides:

$$\left(\frac{I}{R^2} + m\right)s^2Y(s) = -mg\frac{d}{L}\theta(s)$$

$$\text{So, } G(s) = \frac{Y(s)}{\theta(s)} = \frac{-mg\frac{d}{L}}{\left(\frac{I}{R^2}+m\right)s^2}$$

After plugging the values:

$$G(s) = \frac{-(0.11)(-9.8)\frac{(0.03)}{(1.0)}}{\left(\frac{(0.00000999)}{(0.015)^2} + 0.11\right)s^2}$$

$$\text{So, } G(s) = \frac{0.03234}{0.1544s^2} = \frac{0.21}{s^2}$$

Code:

```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;

Y = -m*g*(d/L);
Theta = (s^2)*((I/R^2)+m);

disp('Open Loop Transfer Function G(s)
= Y(s)/Theta(s) of the plant is:');
G = Y/Theta
```

Output:

```
>> FBCS_Project_SP2026
Open Loop Transfer Function G(s) =
Y(s)/Theta(s) of the plant is:

G =

   0.03234
  ----------
  0.1544 s^2

Continuous-time transfer function.
Model Properties

       Numerator: {[0 0 0.0323]}
     Denominator: {[0.1544 0 0]}
        Variable: 's'
         IODelay: 0
      InputDelay: 0
     OutputDelay: 0
       InputName: {''}
       InputUnit: {''}
      InputGroup: [1×1 struct]
      OutputName: {''}
      OutputUnit: {''}
     OutputGroup: [1×1 struct]
           Notes: [0×1 string]
        UserData: []
            Name: ''
```

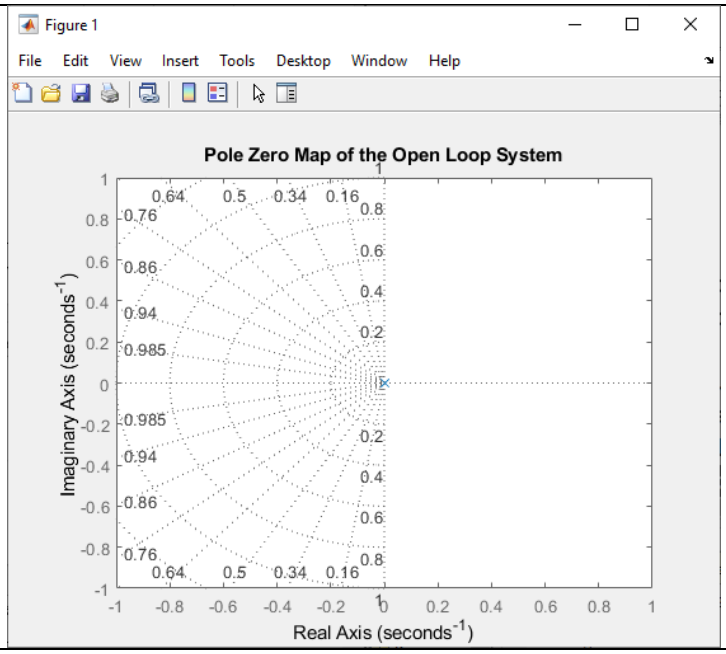
	Ts: 0 TimeUnit: 'seconds' SamplingGrid: [1×1 struct]
Observations:	
Open loop transfer function is $G(s) = 0.21/s^2$, representing a double integrator system.	

b) What are the open loop poles for the above system? Plot the pzmap of the open loop system.

Code:	**Output**

`s = tf('s');`
 	
`alpha = 0;`
 	
`m = 0.11;`
`R = 0.015;`
`d = 0.03;`
`g = -9.8;`
`L = 1.0;`
`I = 0.00000999;`
 	
`Y = -m*g*(d/L);`
`Theta = (s^2)*((I/R^2)+m);`
 	
`G = Y/Theta;`
 	
`p = pole(G);`
`disp('Open Loop Poles of the System are:');`
`disp(p);`
 	
`figure;`
`pzmap(G);`
`title('Pole Zero Map of the Open Loop System');`
`grid on;`
	`>> FBCS_Project_SP2026`
	`Open Loop Poles of the System are:`
	`0`
	`0`

Plot:	



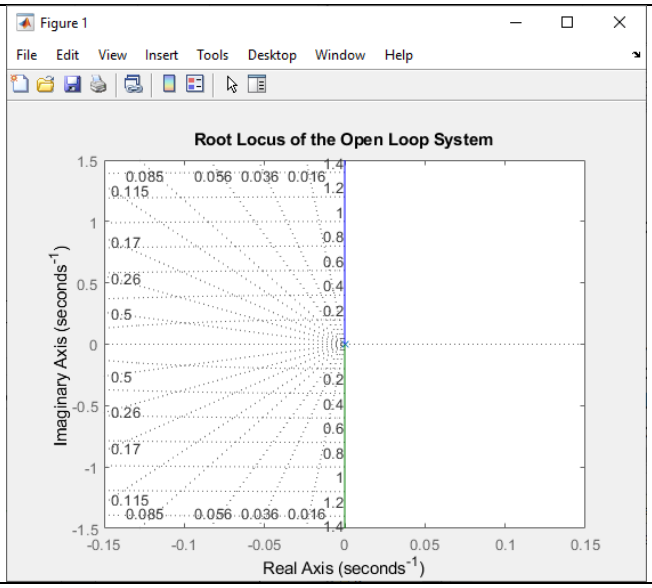
Observations:
The open loop system has two poles at the origin (s = 0, 0). The pole zero map confirms both poles lie at the origin with no zeros present.

c) Plot the root locus of the open loop system and comment if the desired system will be able to meet the *design specifications* by varying the parameter K.

Code:

```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;

Y = -m*g*(d/L);
Theta = (s^2)*((I/R^2)+m);

G = Y/Theta;

figure;
rlocus(G);
title('Root Locus of the Open Loop System');
grid on;
```

Plot:



Observations:
The root locus lies entirely on the imaginary axis, showing no damping as gain (K) varies. Thus, the system cannot meet the design specifications (overshoot and settling time) using only gain adjustment and requires a proper controller.

d) Plot the open loop step response using $0.25 * u(t)$ as input. Comment on the stability of the system. (Hint: In Matlab: step(0.25*sys_ol)) command will provide a step reference of 0.25. What are the characteristics:

i. Overshoot,
ii. Rise time,
iii. Settling time,
iv. Steady-state value

Does it meet the given specifications?

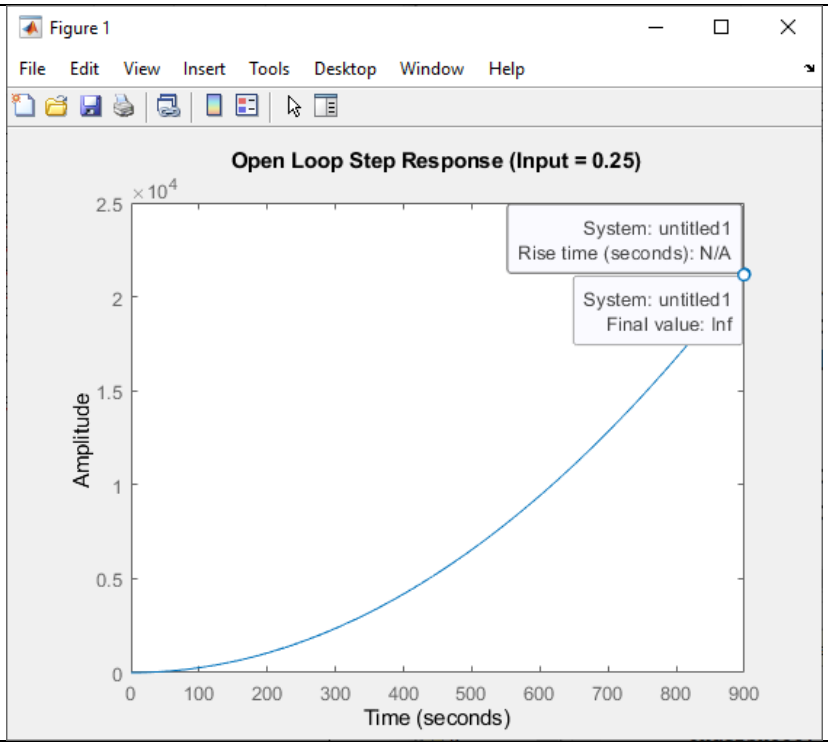
Code:

```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;

Y = -m*g*(d/L);
Theta = (s^2)*((I/R^2)+m);

G = Y/Theta;

R = 0.25;

figure;
step(R * G);
title('Open Loop Step Response (Input = 0.25)');

S = stepinfo(R * G);
disp('Step Response Characteristics:');
disp(S);
```

Output:

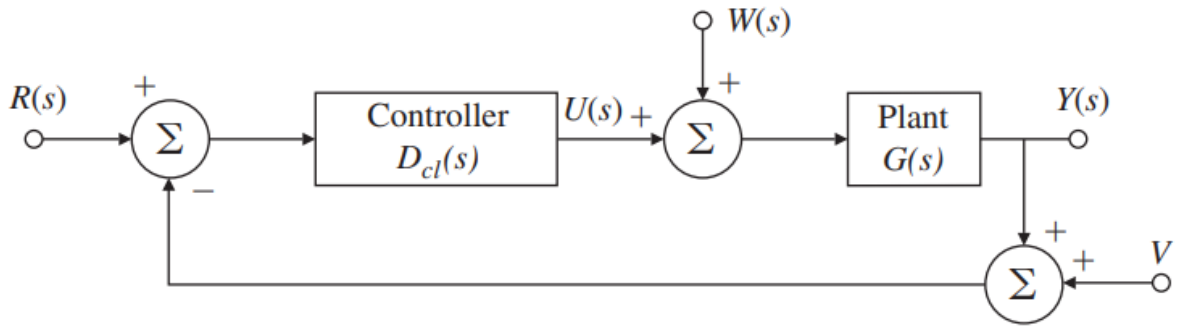
```
>> FBCS_Project_SP2026
Step Response Characteristics:
         RiseTime: NaN
    TransientTime: NaN
     SettlingTime: NaN
      SettlingMin: NaN
      SettlingMax: NaN
        Overshoot: NaN
       Undershoot: NaN
             Peak: Inf
         PeakTime: Inf
```

Plot:



Observations:
The open loop step response is unbounded (output increases indefinitely), indicating the system is unstable. Overshoot, rise time, and settling time are undefined (NaN), steady-state value is infinite, so the system does not meet any design specifications.

e) The closed loop system can be achieved by unity feedback. Consider the following unity feedback architecture for our system.



The following code entered in the MATLAB command window generates the closed-loop transfer function assuming the unity-feedback architecture above and a unity-gain controller $D_{cl}(s) = 1$.

```
sys_cl = feedback(sys_ol,1)
```

Make the closed-loop unity feedback system and plot its step response. What are the characteristics?

v. Overshoot,
vi. Rise time,
vii. Settling time,
viii. Steady-state value

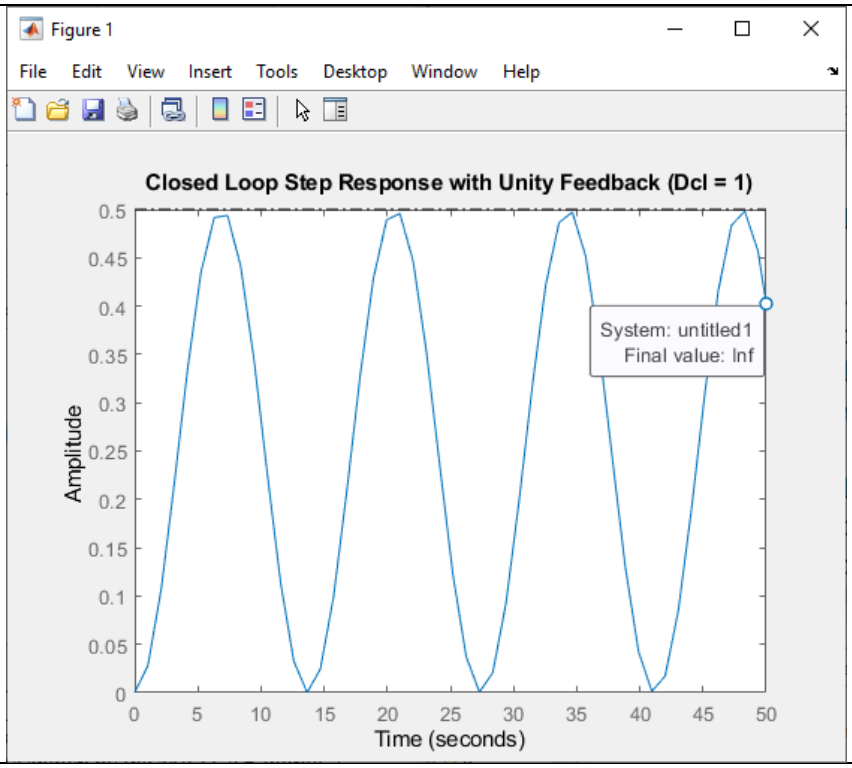
Does it meet the closed-loop specifications?

Code:	**Output:**

`s = tf('s');`
 	
`alpha = 0;`
 	
`m = 0.11;`
`R = 0.015;`
`d = 0.03;`
`g = -9.8;`
`L = 1.0;`
`I = 0.00000999;`
 	
`Y = -m*g*(d/L);`
`Theta = (s^2)*((I/R^2)+m);`
 	
`G = Y/Theta;`
 	
`G_cl = feedback(G, 1);`
 	
`figure;`
	`>> FBCS_Project_SP2026`
	`Closed Loop Step Response Characteristics:`
	`RiseTime: NaN`
	`TransientTime: NaN`
	`SettlingTime: NaN`
	`SettlingMin: NaN`
	`SettlingMax: NaN`
	`Overshoot: NaN`
	`Undershoot: NaN`
	`Peak: Inf`
	`PeakTime: Inf`

```
step(0.25 * G_cl);
title('Closed Loop Step Response with
Unity Feedback (Dcl = 1)');
xlim([0 50]);

S_cl = stepinfo(0.25 * G_cl);
disp('Closed Loop Step Response
Characteristics:');
disp(S_cl);
```

Plot:



Observations:

The closed loop system remains unstable, as the response grows without bound even with unity feedback. All characteristics (overshoot, rise time, settling time) are undefined and steady-state value is infinite, so it does not satisfy the specifications.

f) What are the closed loop poles for the above system? Plot the pzmap of the closed loop system.

Code:

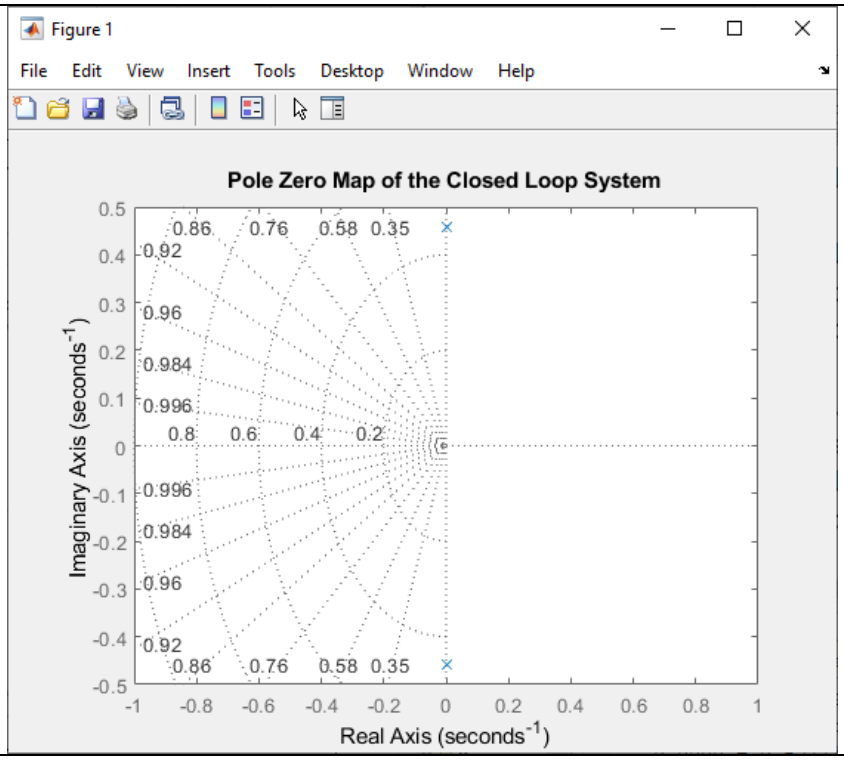
```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;

Y = -m*g*(d/L);
Theta = (s^2)*((I/R^2)+m);
```

Output:

```
>> FBCS_Project_SP2026
Closed Loop Poles of the System are:
   0.0000 + 0.4577i
   0.0000 - 0.4577i
```

```
G = Y/Theta;

G_cl = feedback(G, 1);

p_cl = pole(G_cl);
disp('Closed Loop Poles of the System
are:');
disp(p_cl);

figure;
pzmap(G_cl);
title('Pole Zero Map of the Closed Loop
System');
grid on;
```

Plot:



Observations:

The closed loop poles are purely imaginary at ($s = \pm j0.4577$). This indicates a marginally stable system with sustained oscillations, as shown in the pole zero map.

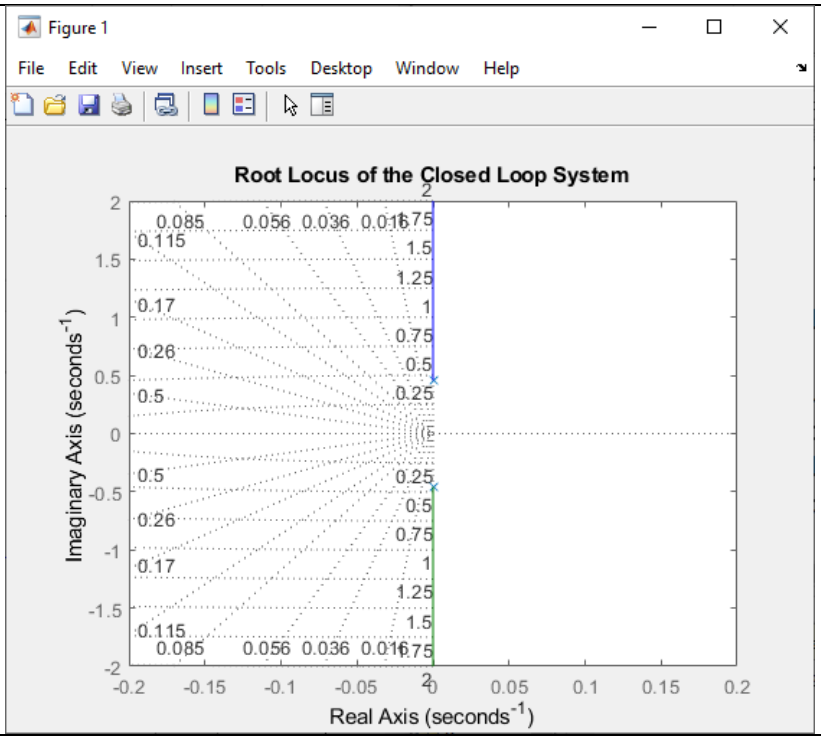
g) Plot the root locus of the closed loop system and comment if the desired system will be able to meet the *design specifications* by varying the parameter K.

Code:

```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;
```

```
Y = -m*g*(d/L);
Theta = (s^2)*((I/R^2)+m);

G = Y/Theta;

G_cl = feedback(G, 1);

figure;
rlocus(G_cl);
title('Root Locus of the Closed Loop System');
grid on;
```

Plot:



Observations:

Root locus shows the closed loop system poles moving along the imaginary axis without entering the left half plane for any gain (K). Hence, varying K alone cannot stabilize the system or satisfy the design specifications; a proper controller is required.

h) Plot the Bode Plot (hint: use margin command instead of bode) of the open loop system with unity feedback and $D_{cl}(s) = 1$ and comment on the results.
What are the gain and phase margins?

Code:

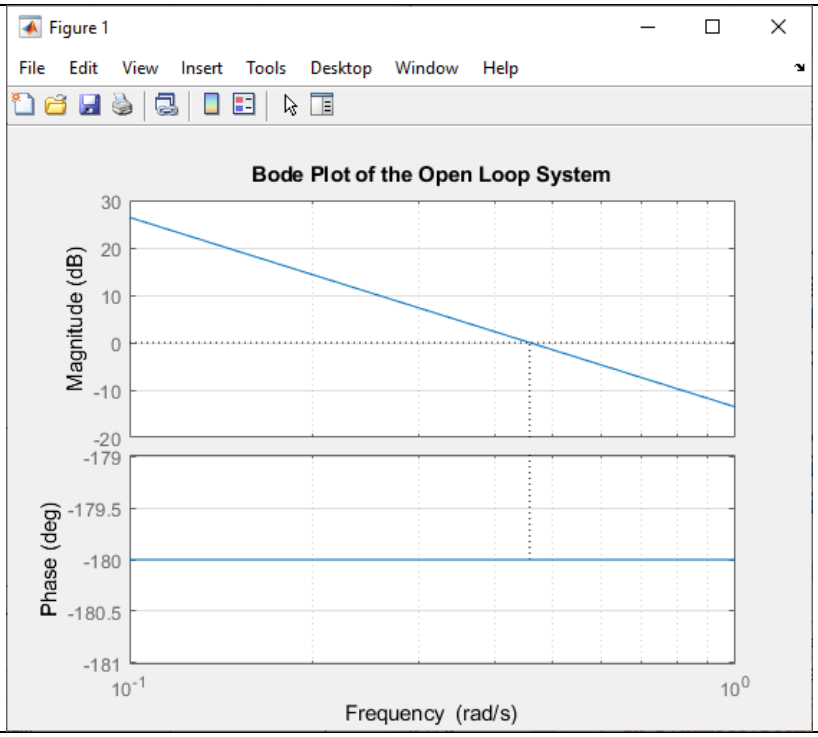
```
s = tf('s');

alpha = 0;

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 0.00000999;

Y = -m*g*(d/L);
```

Output:

```
>> FBCS_Project_SP2026
Gain Margin: 0.000000 dB
Phase Margin: 0.000000 degrees
```

```
Theta = (s^2)*((I/R^2)+m);

G = Y/Theta;

figure;
margin(G);
title('Bode Plot of the Open Loop
System');
grid on;

[gm, pm, wcg, wcp] = margin(G);
fprintf('Gain Margin: %f dB\n',
20*log10(gm));
fprintf('Phase  Margin:  %f  degrees\n',
pm);
```

Plot:



Observations:

Gain margin and phase margin are not properly defined because the system is a double integrator. The system shows no stability margins, indicating it is not stable under unity feedback without compensation.

i) What are your observations from the above results? What kind of controller does this system need? Which poles and/or zeros of the controller will ensure a stable system for this plant?

Observations:

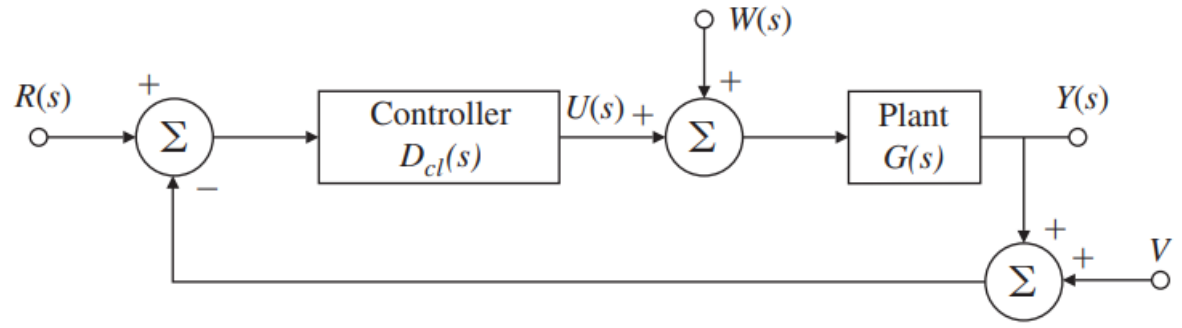
The system behaves like a double integrator and is unstable under unity feedback, so it does not meet time domain specifications. A PD or PID controller is required to add damping and shift the closed loop poles into the left half plane. Introducing a controller zero in the LHP helps stabilize and improve transient response.

Note: Use MATLAB and attach a printout of the results.

Phase 2:

<u>For the Assessment of CLO-3: Recommend a controller including a compensator for an open-loop system to meet closed-loop requirements of stability, performance, and robustness.</u>

Consider the following unity feedback architecture for our system:



[44 Marks]

The following simplified dynamic equation has been linearized about $\alpha = 0$ for a ball and balancer control system:

$$\left(\frac{I}{R^2} + m\right)\ddot{y} = -mg\frac{d}{L}\theta$$

In this system $y(t)$ is the output of the open loop system and $\theta(t) = u(t)$ is the control input to the system. i.e. $G(s) = \frac{Y(s)}{\theta(s)}$

A reference input $R(s) = Y_{des}(s) =$ **step of <u>0.25 units</u>** is provided to the closed loop system, and the **design specifications** for the overall closed loop system are as follows:

- The overshoot should be less than 5%
- Settling time should be less than 3 seconds for a 2% settling time
- Steady-state error for reference tracking should be less than 2%

Since the unity feedback structure with $D_{cl}(s) = 1$ does not meet the of the closed loop design criteria given above, there is a need to make the response of the closed loop system, better. For that a compensator $(D_C(s))$ with gain parameter (K) is required. i.e. $D_{cl}(s) = KD_C(s)$.

a) First, convert the given specifications into frequency domain specifications i.e. Phase margin, required bandwidth and required cross-over frequency. [8]

Calculations:
To determine ω_c from the settling time (T_s) requirement under an initial assumption of K = 1, we use the standard 2% tolerance criterion relationship: $$T_s = \frac{4}{\omega_c}$$ Given the restriction that $T_s < 3$s: $$3 = \frac{4}{\omega_c} \; or \; \omega_c = \frac{4}{3} \approx 1.33 rad/s$$ To ensure a safe transient design margin, we choose the target cross-over frequency to be: $$\omega_c = 1.5 \text{ rad/s}$$

To satisfy the constraint that overshoot must be less than 5% ($M_p < 5\%$):

$$M_p = e^{\frac{-\pi\zeta^2}{\sqrt{1-\zeta^2}}} \; or \; \zeta = \sqrt{\frac{\ln (M_p)^2}{\pi^2 + \ln (M_p)^2}}$$

$$\zeta = \sqrt{\frac{\ln (0.05)^2}{\pi^2 + \ln (0.05)^2}}$$

$$\zeta \approx 0.7$$

Using the second-order relationship $PM \approx \zeta \text{ x } 100°$:

$$PM_{target} \approx 0.7 \times 100° = 70°$$

Using the standard frequency-domain scaling relationship for a system with a damping ratio of $\zeta \approx 0.7$:

$$\omega_B \approx 1.4 \times \omega_c$$
$$\omega_B = 1.4 \times 1.5 \text{ rad/s}$$
$$\omega_B = 2.1 \text{ rad/s}$$

b) What is the value of K required to meet the ω_c requirement? Find the value and check your results in Matlab by plotting the Bode Plot of $KG(s)$. [8]

Calculations:

The open-loop plant transfer function from Phase 1 is:

$$G(s) = \frac{0.21}{s^2}$$

To establish the gain crossover frequency at $\omega_c = 1.5$ rad/s with an uncompensated loop ($D_c(s) = 1$), the loop magnitude must equal 1 (0 dB) at that frequency:

$$\left| K \frac{0.21}{j\omega_c{}^2} \right| = 1$$

$$\left| K \frac{0.21}{j1.5^2} \right| = 1$$

$$|G(j1.5)| = \frac{0.21}{1.5^2} = \frac{0.21}{2.25} \approx 0.09309$$

Solving for K:

$$K = \frac{1}{0.09309}$$

$$K = 10.74$$

MATLAB Code:

```
s = tf('s');
m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
```

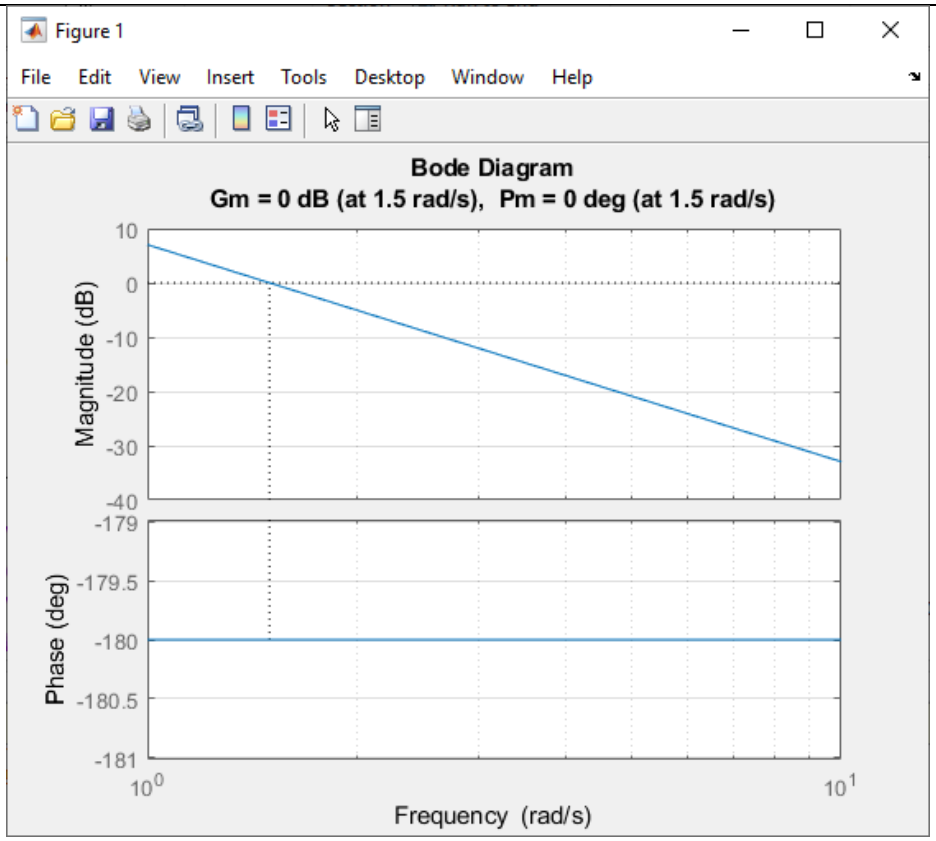
Output:

```
>> FBCS_Project_SP2026
Verification Results
Calculated Loop Gain K: 10.74
Verified Gain Crossover Frequency
(w_c): 1.50 rad/s
```

```
L = 1.0;
I = 9.99e-6;

num_G = -m * g * (d / L);
den_G = (I / (R^2)) + m;
G = num_G / (den_G * s^2);

K = 10.74;
Uncompensated_Loop = K * G;

figure;
margin(Uncompensated_Loop);
grid on;

[~, Phase_Margin, ~, Crossover_Freq] =
margin(Uncompensated_Loop);

fprintf('Verification Results\n');
fprintf('Calculated Loop Gain K:
%.2f\n', K);
fprintf('Verified Gain Crossover
Frequency (w_c): %.2f rad/s\n',
Crossover_Freq);
fprintf('Measured Phase Margin (PM):
%.2f degrees\n', Phase_Margin);
```

```
Measured Phase Margin (PM): 0.00
degrees
```

Plot:



c) Design a Compensator for the plant to meet the required design criteria. [20]

Calculations:

The open loop plant transfer function:

$$G(s) = \frac{0.21}{s^2}$$

To meet the required design criteria, we design a single stage lead compensator in the required structural form:

$$D_c(s) = \frac{\frac{s}{\omega_d} + 1}{\frac{s}{\omega_i} + 1}$$

$$\emptyset_{max} = 70° - 0° + 0° = 70°$$

The uncompensated double integrator system has a constant phase margin of 0°. To meet the target specification exactly, the lead stage must provide a maximum phase lift ($\emptyset_{max}$) of 70°:

$$\propto = \frac{1 - \sin(70°)}{1 + \sin(70°)} = \frac{1 - 0.9397}{1 + 0.9397} = \frac{0.0603}{1.9397} = 0.0311$$

$$Lead = \frac{1}{\propto} = \frac{1}{0.0311} = 32.15$$

The maximum phase contribution frequency is matched to our chosen target crossover frequency, $\omega_{max} = \omega_c = 1.5 rad/s$:

$$zero = \omega_d = \omega_{max}\sqrt{\propto} = (1.5)\left(\sqrt{0.0311}\right) = 0.2645 rad/s$$

$$pole = \omega_i = \frac{\omega_{max}}{\sqrt{\propto}} = \frac{(1.5)}{\left(\sqrt{0.0311}\right)} = 8.5058 rad/s$$

Now substituting these values in the required structural format:

$$D_c(s) = \frac{\frac{s}{0.2645} + 1}{\frac{s}{8.5058} + 1}$$

To ensure that $\omega_c = 1.5 rad/s$ remains the true loop crossover point, the combined magnitude of the gain, compensator, and plant must equal 1 (0dB) at this frequency

$$|KD_c(j1.5)G(1.5)| = 1$$

Evaluating magnitudes at s = j1.5:

$$|G(j1.5)| = \frac{0.21}{1.5^2} = 0.09309$$

$$|D_c(j1.5)| = \left|\frac{\frac{j1.5}{0.2645} + 1}{\frac{j1.5}{8.5058} + 1}\right| = \frac{\sqrt{1^2 + 5.6711^2}}{\sqrt{1^2 + 0.1763^2}} = \frac{5.7585}{1.0154} = 5.6711$$

Solving for the scalar gain K:

$$K = \frac{1}{(5.6711)(0.09309)} = 1.894$$

Multiplying K into the structure yields the complete, project-ready controller expression:

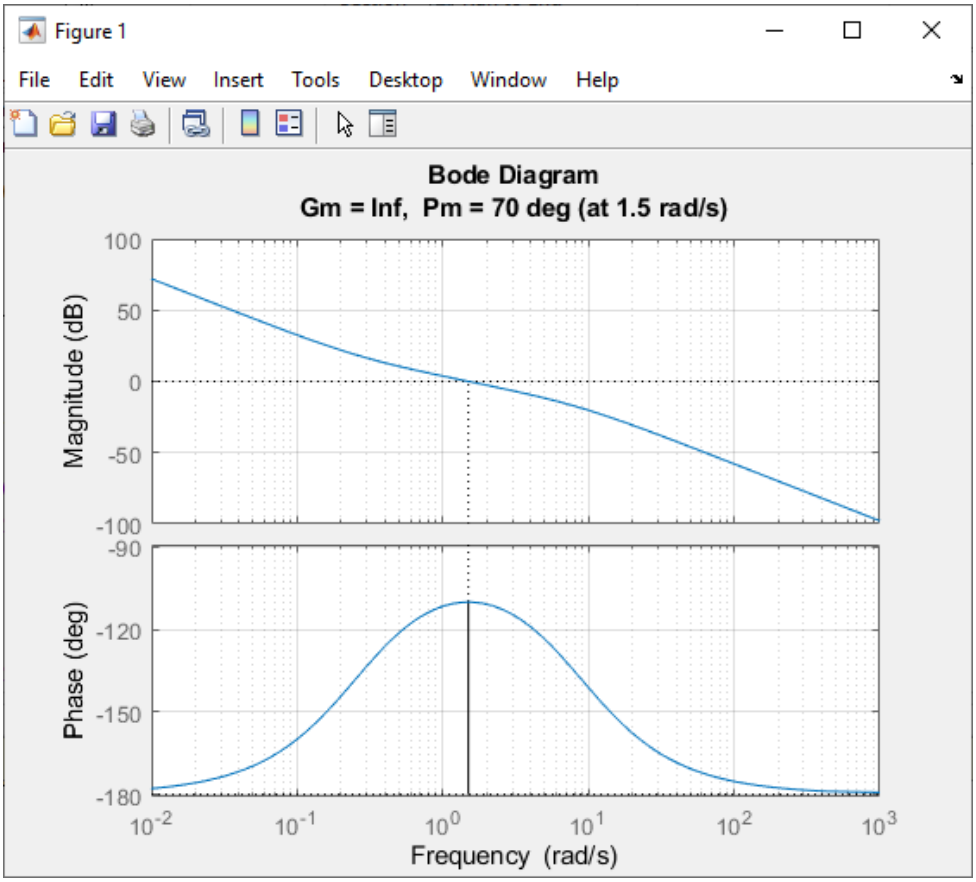
$$D_{cl}(s) = (1.894)\left(\frac{\frac{s}{0.2645}+1}{\frac{s}{8.5058}+1}\right)$$

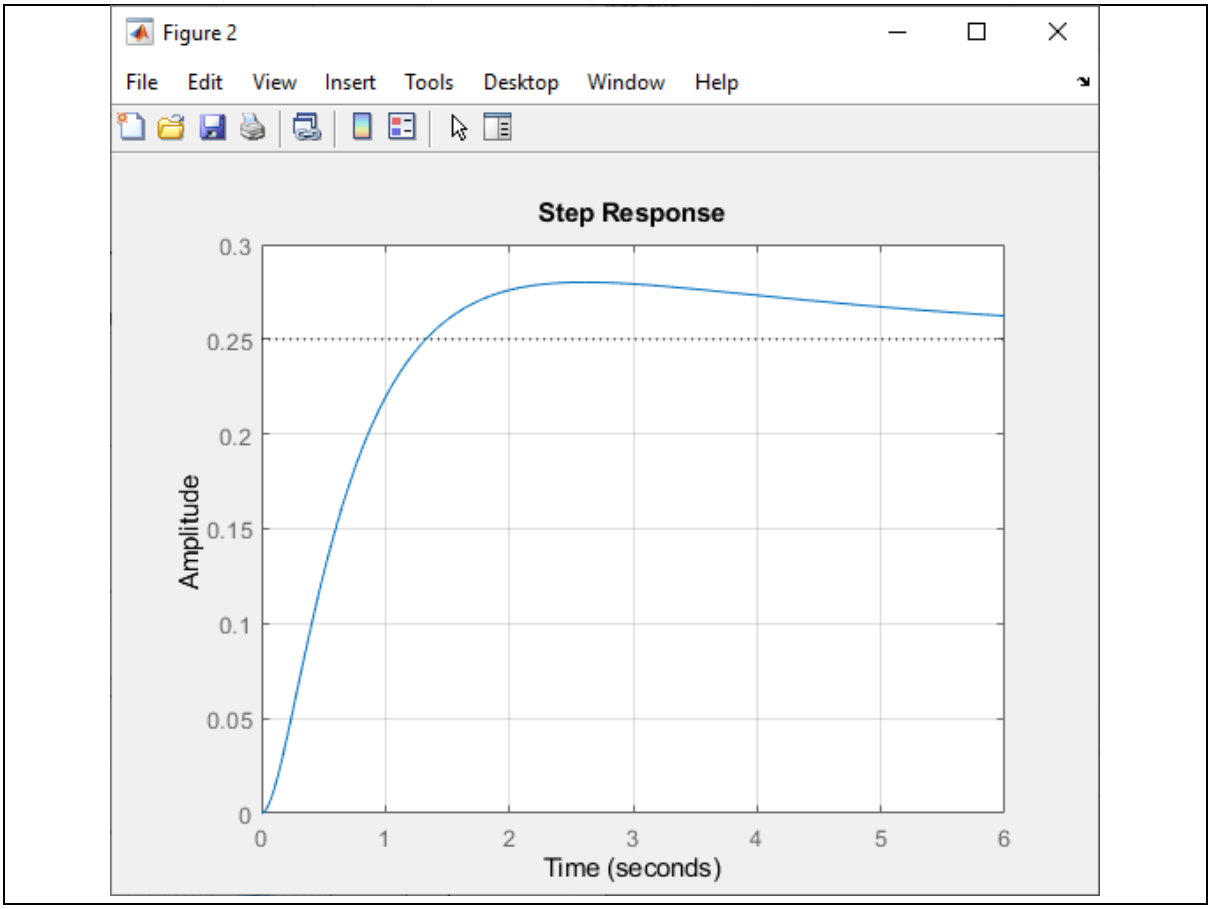
Code:

```
s = tf('s');

m = 0.11;
R = 0.015;
d = 0.03;
g = -9.8;
L = 1.0;
I = 9.99e-6;

num_G = -m * g * (d / L);
den_G = (I / (R^2)) + m;
G = num_G / (den_G * s^2);

alpha = 0.0311;
w_max = 1.5;
w_d = w_max * sqrt(alpha);
w_i = w_max / sqrt(alpha);

G_mag = abs(evalfr(G, 1i * w_max));
D_norm = ((1i * w_max / w_d) + 1) / ((1i * w_max / w_i) + 1);
K = 1 / (abs(D_norm) * G_mag);

D_cl = K * ((s / w_d) + 1) / ((s / w_i) + 1);

fprintf('Designed Lead Compensator Dcl(s):\n');
display(D_cl);

Loop_TF = D_cl * G;
figure();
margin(Loop_TF);
grid on;

[~, Pm, ~, Crossover_Freq] = margin(Loop_TF);

System_CL = feedback(Loop_TF, 1);
step_input = 0.25;
t_axis = 0:0.01:6;

figure();
step(step_input * System_CL, t_axis);
grid on;

perf_stats = stepinfo(System_CL, 'SettlingTimeThreshold', 0.02);

[y_out, ~] = step(System_CL, t_axis);
ess = abs((1 - y_out(end)) / 1) * 100;

fprintf('\nPERFORMANCE\n');
fprintf('%-25s | %-12s | %-12s | %-12s | %-12s\n', 'Design Stage', 'Crossover
Wc', 'Phase Margin', 'Overshoot (%)', 'Settling Time (s)');
fprintf('%-25s | %-12.2f | %-12.2f | %-12.2f | %-12.2f\n', 'Compensator Part
(c)', Crossover_Freq, Pm, perf_stats.Overshoot, perf_stats.SettlingTime);
fprintf('\n\n');
```

Output and Plot:

```
>> FBCS_Project_SP2026
Designed Lead Compensator Dcl(s):

D_cl =

  16.11 s + 4.262
  ---------------
  0.2645 s + 2.25

Continuous-time transfer function.
Model Properties

PERFORMANCE
Design Stage              | Crossover Wc | Phase Margin | Overshoot (%) | Settling
Time (s)
Compensator Part (c)      | 1.50         | 70.00        | 12.03         | 8.68
```





d) Iterate on the design to achieve all the design specifications. [8]

Code:

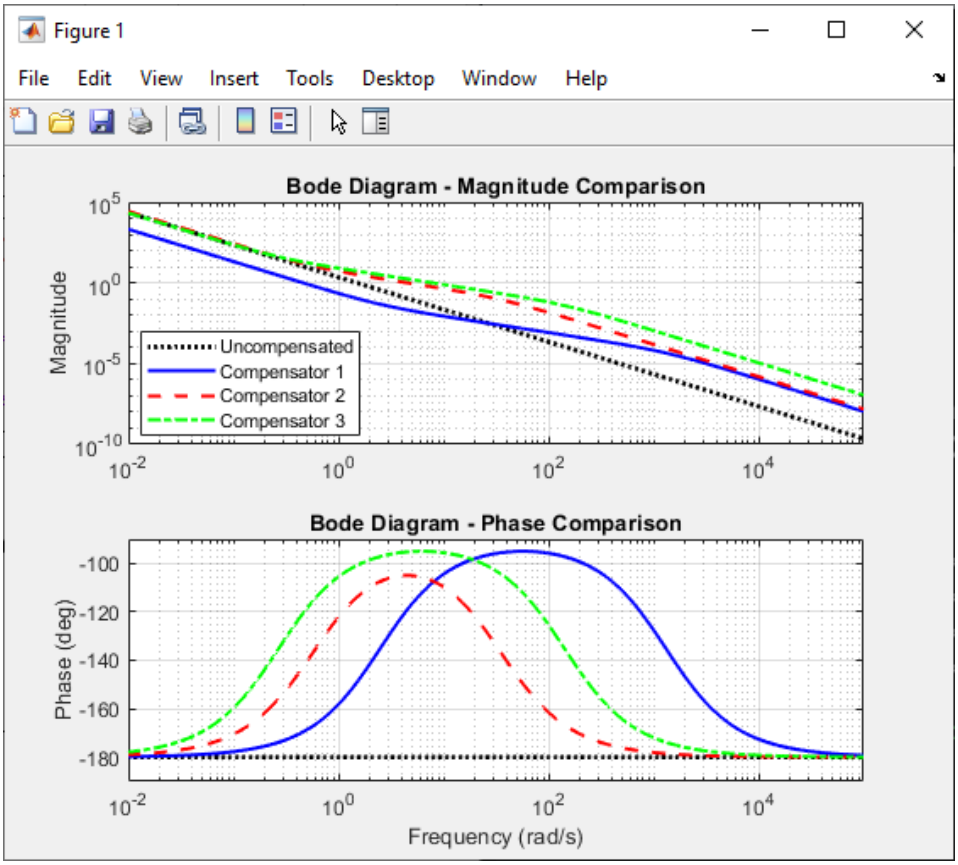
```
num_G = [0.2095];
den_G = [1 0 0];
sys_G = tf(num_G, den_G);
step_amplitude = 0.25;

K_uncomp = 10.74;
sys_L0 = K_uncomp * sys_G;
sys_T0 = feedback(sys_L0, 1);

num_C1 = 524.6 * [1 2.5]; den_C1 = [1 1311];
sys_C1 = tf(num_C1, den_C1);
sys_L1 = series(sys_C1, sys_G); sys_T1 = feedback(sys_L1, 1);

num_C2 = 741 * [1 0.59]; den_C2 = [1 34.2];
sys_C2 = tf(num_C2, den_C2);
sys_L2 = series(sys_C2, sys_G); sys_T2 = feedback(sys_L2, 1);

num_C3 = 5354 * [1 0.2707]; den_C3 = [1 142];
sys_C3 = tf(num_C3, den_C3);
sys_L3 = series(sys_C3, sys_G); sys_T3 = feedback(sys_L3, 1);

[~, Pm0, ~, ~] = margin(sys_L0); info0 = stepinfo(sys_T0);
[~, Pm1, ~, ~] = margin(sys_L1); info1 = stepinfo(sys_T1);
[~, Pm2, ~, ~] = margin(sys_L2); info2 = stepinfo(sys_T2);
[~, Pm3, ~, ~] = margin(sys_L3); info3 = stepinfo(sys_T3);
```

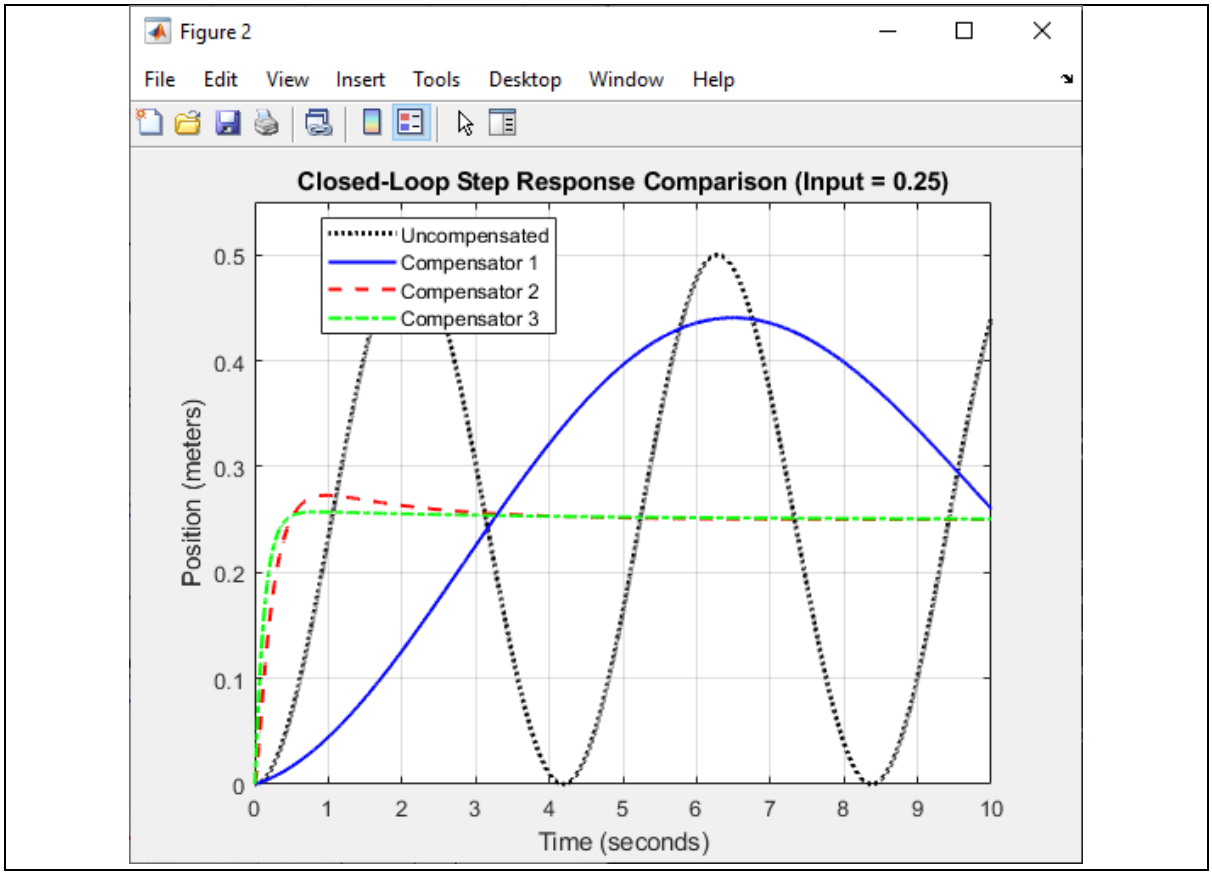
```
fprintf('\nPERFORMANCE\n');
fprintf('%-25s | %-12s | %-12s | %-12s\n', 'Design Stage', 'Phase Margin',
'Overshoot (%)', 'Settling Time (s)');
fprintf('\n');
fprintf('%-25s | %-12.2f | %-12.2f | %-12.2f\n', 'Uncompensated Loop', Pm0,
info0.Overshoot, info0.SettlingTime);
fprintf('%-25s | %-12.2f | %-12.2f | %-12.2f\n', 'Compensator 1', Pm1,
info1.Overshoot, info1.SettlingTime);
fprintf('%-25s | %-12.2f | %-12.2f | %-12.2f\n', 'Compensator 2', Pm2,
info2.Overshoot, info2.SettlingTime);
fprintf('%-25s | %-12.2f | %-12.2f | %-12.2f\n', 'Compensator 3', Pm3,
info3.Overshoot, info3.SettlingTime);
fprintf('\n\n');

w_vec = logspace(-2, 5, 2000);

[mag0, phase0] = bode(sys_L0, w_vec);
[mag1, phase1] = bode(sys_L1, w_vec);
[mag2, phase2] = bode(sys_L2, w_vec);
[mag3, phase3] = bode(sys_L3, w_vec);

figure();
subplot(2,1,1);
loglog(w_vec, squeeze(mag0), 'k:', 'LineWidth', 2.0);
hold on;
loglog(w_vec, squeeze(mag1), 'b-', 'LineWidth', 1.5);
loglog(w_vec, squeeze(mag2), 'r--', 'LineWidth', 1.5);
loglog(w_vec, squeeze(mag3), 'g-.', 'LineWidth', 1.5);
grid on;
xlim([10^-2, 10^5]);
title('Bode Diagram - Magnitude Comparison');
ylabel('Magnitude');
legend('Uncompensated', 'Compensator 1', 'Compensator 2', 'Compensator 3',
'Location', 'best');

subplot(2,1,2);
semilogx(w_vec, squeeze(phase0), 'k:', 'LineWidth', 2.0);
hold on;
semilogx(w_vec, squeeze(phase1), 'b-', 'LineWidth', 1.5);
semilogx(w_vec, squeeze(phase2), 'r--', 'LineWidth', 1.5);
semilogx(w_vec, squeeze(phase3), 'g-.', 'LineWidth', 1.5);
grid on;
xlim([10^-2, 10^5]);
ylim([-190, -90]);
title('Bode Diagram - Phase Comparison');
xlabel('Frequency (rad/s)');
ylabel('Phase (deg)');

figure();
t_axis = 0:0.002:10;

[y0, ~] = step(sys_T0, t_axis);
[y1, ~] = step(sys_T1, t_axis);
[y2, ~] = step(sys_T2, t_axis);
[y3, ~] = step(sys_T3, t_axis);

plot(t_axis, y0 * step_amplitude, 'k:', 'LineWidth', 2.0); hold on;
plot(t_axis, y1 * step_amplitude, 'b-', 'LineWidth', 1.5);
```

```
plot(t_axis, y2 * step_amplitude, 'r--', 'LineWidth', 1.5);
plot(t_axis, y3 * step_amplitude, 'g-.', 'LineWidth', 1.5);
grid on;
xlim([0, 10]);
ylim([0, 0.55]);
title('Closed-Loop Step Response Comparison (Input = 0.25)');
xlabel('Time (seconds)');
ylabel('Position (meters)');
legend('Uncompensated', 'Compensator 1', 'Compensator 2', 'Compensator 3',
'Location', 'best');
```

Output and Plot:

```
>> FBCS_Project_SP2026

PERFORMANCE
Design Stage          | Phase Margin | Overshoot (%) | Settling Time (s)

Uncompensated Loop    | 0.00         | NaN           | NaN
Compensator 1         | 10.44        | 75.44         | 90.46
Compensator 2         | 75.03        | 9.14          | 3.41
Compensator 3         | 84.85        | 2.89          | 2.30
```





Note: Use MATLAB and attach a printout of the results.

Report – Ball and Balancer Control System

Introduction and Project Overview

The Ball and Balancer system is a common control engineering problem used to study unstable and nonlinear systems. The objective is to keep a ball at a desired position on a beam by adjusting the beam's angle using a motor. Because the ball naturally rolls away from its position, an effective controller is required to maintain stability and accurate positioning.
This project is divided into two main phases:

1. **Phase 1 – System Modeling and Analysis:** Developing the mathematical model of the system, studying its open-loop behavior, and examining its response with simple unity feedback.
2. **Phase 2 – Controller Design:** Converting the required time-domain specifications into frequency-domain requirements and designing a suitable compensator to achieve the desired performance.

Design Requirements

The control system is tested using a step position command. The system must satisfy the following performance requirements:

- **Maximum Overshoot:** The ball should not move too far beyond the desired position.
- **Settling Time:** The ball should reach and remain near the target position as quickly as possible.
- **Steady-State Error:** The final position of the ball should closely match the reference position with little or no error.

Phase 1: System Modeling and Analysis

Mathematical Model

The behavior of the Ball and Balancer system is described using Newton’s laws of motion. The movement of the ball and the rotation of the beam are combined to derive the governing equations of the system.
To simplify controller design, the nonlinear equations are linearized around the equilibrium position where the beam angle is small. Applying the Laplace Transform converts the equations into a transfer function that relates the beam angle (input) to the ball position (output).
The resulting transfer function behaves as a double integrator multiplied by a constant gain. This gain depends on physical parameters such as the ball mass, radius, beam dimensions, and moment of inertia.

Stability Analysis

The open-loop system has two poles located at the origin of the s-plane. This indicates that the plant has no natural damping and is difficult to control.
Key observations include:

- **Open-Loop Behavior:** A step input causes the ball position to continuously increase, showing that the system cannot regulate itself.

- **Unity Feedback Response:** Adding simple unity feedback moves the poles to the imaginary axis, resulting in continuous oscillations. The ball keeps moving back and forth without settling at the desired position.

These results show that the system cannot be controlled effectively without a properly designed compensator.

Phase 2: Controller Design

Converting Specifications to Frequency Domain

The time-domain requirements are converted into frequency-domain targets to facilitate controller design.

- The settling-time requirement determines the desired crossover frequency.
- The overshoot requirement is used to calculate the required damping ratio and phase margin.
- The desired system speed helps estimate the required closed-loop bandwidth.

Determining the Required Gain

The first step is to find a gain value that places the gain crossover frequency at the required location.

Although increasing the gain can achieve the desired crossover frequency, it does not improve stability. Since the plant behaves like a double integrator, it introduces approximately $-180°$ of phase lag. As a result, the phase margin remains extremely low, indicating that gain adjustment alone cannot meet the design requirements.

Therefore, additional phase compensation is necessary.

Lead Compensator Design

A phase-lead compensator is introduced to provide the extra phase margin required for stability.

The compensator contains a zero and a pole. The zero is placed at a lower frequency, while the pole is located at a higher frequency. This arrangement increases the phase margin around the crossover frequency.

Although the initial lead compensator satisfies the frequency-domain requirements, the time-domain response still shows excessive overshoot and slower settling than expected.

This occurs because the compensator introduces a zero that increases the transient response, causing additional peaking and oscillations.

Iterative Controller Improvement

Several compensator configurations were tested to improve performance:

- **Compensator 1:** Stabilized the system but produced large overshoot and noticeable oscillations.
- **Compensator 2:** Reduced overshoot and improved damping, but the settling time remained slightly above the required limit.
- **Compensator 3:** Increased the phase margin and bandwidth further, resulting in low overshoot, faster settling, and improved overall performance.

The third compensator was selected because it met all design specifications.

System Performance

Dynamic Response

The results demonstrate the relationship between controller gain, damping, and response speed.

- The uncompensated system is unstable.
- Compensator 1 provides stability but insufficient damping.
- Compensator 2 improves damping but is still relatively slow.
- Compensator 3 achieves the best balance between speed and stability.

The final design produces a smooth response with minimal oscillations and rapid settling.

Steady-State Accuracy

The Ball and Balancer plant behaves as a Type-2 system because it contains two integrators. According to steady-state error theory, a Type-2 system can track a step input with zero steady-state error. As a result, the ball eventually reaches the desired position without any permanent offset.

Challenges and Solutions

Several issues were encountered during the project:

- **Discontinuous Frequency Response Curves:** This problem was caused by inconsistent frequency vectors and was solved by using a common logarithmic frequency range.
- **Incorrect Settling-Time Measurements:** A scaling error in the simulation model affected the results. Separating the system dynamics from the input scaling corrected the issue.
- **Excessive Transient Peaks:** The first lead compensator introduced significant overshoot. Repeated tuning of the compensator parameters reduced the transient peak and improved performance.

Conclusion

The Ball and Balancer system is naturally unstable and cannot be controlled effectively using only gain adjustment or simple feedback.

A phase-lead compensator was designed to improve stability and dynamic performance. Through several design iterations, the controller parameters were refined until all performance requirements were satisfied.

The final controller provides stable operation, low overshoot, fast settling time, and zero steady-state error, making it suitable for accurate ball position control on the balancing platform.